1. Use LinkedHashMap When Insertion Order Matters (Interview Notes)

What is LinkedHashMap?

LinkedHashMap is an implementation of the **Map** interface that stores data as **key-value pairs** while **preserving the insertion order**.

It combines the features of **HashMap** and a **doubly linked list**.

```
LinkedHashMap map = new LinkedHashMap<>();

map.put(3, "C");
map.put(1, "A");
map.put(2, "B");

System.out.println(map);
```

Output

```
{3=C, 1=A, 2=B}
```

The order is exactly the same as insertion.

Why Use LinkedHashMap?

Use LinkedHashMap when:

- Fast lookup is needed.
- The order of insertion must be preserved.
- Predictable iteration order is important.

Examples:

- Recently viewed products
- Browser history
- Caching (LRU Cache using access order)

Important Interview Points ⭐

- LinkedHashMap implements the **Map** interface.
- Maintains **insertion order** using a **doubly linked list**.
- Provides **O(1) average** time complexity for put(), get(), and remove().
- Allows **one null key** and **multiple null values**.
- Slightly slower and uses more memory than HashMap because of the linked list.
- Can also maintain **access order** (used in LRU cache) by using a special constructor.

Tricky Interview Questions

Q1. Difference between HashMap and LinkedHashMap?

Answer:

- HashMap → No guaranteed order.
- LinkedHashMap → Preserves insertion order.

Q2. Why is LinkedHashMap slightly slower?

Answer:

Because it maintains an additional **doubly linked list** to preserve order.

Q3. Does LinkedHashMap allow duplicate keys?

Answer: No.

Duplicate keys replace the previous value.

Q4. Can LinkedHashMap maintain access order?

Answer: Yes.

Using its constructor with accessOrder = true, it can reorder entries based on recent access, which is useful for implementing **LRU caches**.

Interview Summary ⭐

- **LinkedHashMap preserves insertion order while providing fast key-based lookup.**
- Uses a **hash table + doubly linked list** internally.
- Average complexity for put(), get(), and remove() is **O(1)**.
- Uses slightly more memory than HashMap.
- Best choice when **both fast lookup and predictable iteration order** are required.

> **Interview One-Liner:**
>
> **"LinkedHashMap provides HashMap's fast lookup while preserving insertion order using a doubly linked list."**

---

2. Use TreeMap When Sorted Keys Matter (Interview Notes)

What is TreeMap?

TreeMap is an implementation of the **Map** interface that stores key-value pairs in **sorted order of keys**.

It automatically sorts keys using their **natural ordering** or a **Comparator**.

```
TreeMap map = new TreeMap<>();

map.put(30, "C");
map.put(10, "A");
map.put(20, "B");

System.out.println(map);
```

Output

```
{10=A, 20=B, 30=C}
```

Why Use TreeMap?

Use TreeMap when:

- Keys must always remain sorted.
- Range-based operations are required.
- Ordered traversal is important.

Examples:

- Leaderboards
- Rankings
- Dictionary
- Employee records sorted by ID

Important Interview Points ⭐

- TreeMap implements the **Map** interface.
- Stores entries sorted by **key**, not value.
- Internally implemented using a **Red-Black Tree**.
- Operations (put(), get(), remove()) take **O(log n)** time.
- Does **not allow null keys**.
- Keys must implement **Comparable** or a **Comparator** must be provided.
- Values can be duplicated.

Tricky Interview Questions

Q1. Why is TreeMap slower than HashMap?

Answer:

Because it maintains sorted order using a **Red-Black Tree**, resulting in **O(log n)** operations.

Q2. Does TreeMap sort values?

Answer: No.

It sorts **keys only**.

Q3. Can TreeMap store null keys?

Answer: No.

A NullPointerException is thrown because keys must be comparable.

Q4. When should TreeMap be preferred?

Answer:

When sorted keys or range queries are required.

Interview Summary ⭐

- **TreeMap stores key-value pairs in sorted key order.**
- Uses a **Red-Black Tree** internally.
- Provides **O(log n)** performance for major operations.
- Does **not allow null keys**.
- Best suited for applications requiring **sorted keys**.

> **Interview One-Liner:**
>
> **"TreeMap automatically keeps keys sorted using a Red-Black Tree, making it ideal when ordered keys are required."**

---

3. Analyze HashMap Average Time Complexity for put() and get() (Interview Notes)

Time Complexity

Operation	Average	Worst Case
put(key, value)	**O(1)**	**O(n)** *(O(log n) for heavily-collided buckets in modern Java)*
get(key)	**O(1)**	**O(n)** *(O(log n) after treeification)*

Why is put() O(1)?

```
map.put(101, "Ali");
```

Steps:

1. Compute hashCode()
2. Find bucket
3. Insert the key-value pair

Most of the time, no searching is needed.

Why is get() O(1)?

```
map.get(101);
```

Steps:

1. Compute hashCode()
2. Find bucket
3. Compare keys using equals()
4. Return the value

Lookup is very fast because it goes directly to the bucket.

Important Interview Points ⭐

- HashMap provides **O(1) average** time for put() and get().
- Performance depends on a **good distribution of hash codes**.
- Excessive collisions increase lookup time.
- In modern Java (Java 8+), buckets with many collisions can become **Red-Black Trees**, improving worst-case performance.
- Choosing good hashCode() implementations is important for maintaining O(1) average performance.

Tricky Interview Questions

Q1. Why is HashMap O(1) on average?

Answer:

Because hashing allows direct access to the correct bucket.

Q2. Why isn't HashMap always O(1)?

Answer:

Multiple collisions require additional comparisons.

Q3. What is the worst-case complexity?

Answer:

- **O(n)** in older implementations.
- **O(log n)** for treeified buckets in modern Java.

Q4. Does a poor hashCode() affect performance?

Answer: Yes.

Poor hash distribution increases collisions and slows operations.

Interview Summary ⭐

- put() and get() are **O(1) average** because hashing directly locates buckets.
- Performance depends on **efficient hashing and minimal collisions**.
- Heavy collisions reduce performance.
- Modern Java improves worst-case performance using **Red-Black Trees**.

> **Interview One-Liner:**
>
> **"HashMap achieves O(1) average lookup and insertion by using hash codes to directly locate buckets."**

---

4. Interview Insight: Explain How HashMap Finds a Value Using a Key Hash

How Does HashMap Find a Value?

Suppose we have:

```
HashMap map = new HashMap<>();

map.put(101, "Ali");
```

Now,

```
map.get(101);
```

Internally, HashMap performs these steps.

Step-by-Step Process

Step 1: Calculate the Hash Code

```
101.hashCode()
```

↓

Generates a hash value.

Step 2: Find the Bucket

The hash value is converted into a bucket index.

```
Hash Code

↓

Bucket 5
```

Step 3: Search Inside the Bucket

If only one entry exists:

```
Bucket 5

101 → Ali
```

Return the value immediately.

If multiple entries exist (collision):

```
Bucket 5

101 → Ali

201 → Ahmed
```

HashMap compares keys using **equals()** until it finds the correct one.

Step 4: Return the Value

```
Ali
```

Visual Flow

```
Key (101)

↓

hashCode()

↓

Bucket Index
```

```
↓

Correct Bucket

↓

equals()

↓

Return Value
```

Important Interview Points ⭐

- HashMap first computes the key's **hashCode()**.
- The hash code determines the **bucket location**.
- If multiple entries exist in the bucket, **equals()** identifies the correct key.
- The value associated with the matching key is returned.
- This combination of **hashCode() + equals()** enables fast average-time lookups.

Tricky Interview Questions

Q1. Which method is called first?

Answer:

hashCode().

Q2. When is equals() called?

Answer:

Only when multiple candidate keys exist in the same bucket.

Q3. Why are both hashCode() and equals() needed?

Answer:

- hashCode() locates the bucket.
- equals() identifies the exact key.

Q4. What happens if the key is not found?

Answer:

get() returns null.

Q5. What happens if the key's hash code changes after insertion?

Answer:

The entry remains in its original bucket, and future lookups may fail because the new hash code points to a different bucket.

Interview Summary ⭐

- HashMap finds values by **calculating the key's hash code**.
- The hash code determines the **bucket** containing the entry.
- If collisions occur, equals() is used to locate the exact key.
- This design enables **O(1) average** lookup performance.
- Understanding the sequence **hashCode() → bucket → equals() → value** is one of the most common Java collection interview questions.

> **Interview One-Liner:**
>
> **"When get(key) is called, HashMap computes the key's hash code, locates the appropriate bucket, uses equals() if necessary, and returns the value associated with the matching key."**